Actividad Grupal · Patrones GoF

Modelación y Diseño de Sistemas · MyD 2026 · UNC-ESCMB-AUSI Bloque 2 · Diseño · Complementaria a la clase D03

---

1. Objetivos pedagógicos

Al finalizar la actividad, los alumnos deberán ser capaces de:

1. **Reconocer** un problema de diseño OO recurrente y mapearlo al patrón GoF que mejor lo resuelve.
2. **Explicar** la intención, estructura, consecuencias y trampas típicas de al menos un patrón GoF.
3. **Aplicar** el patrón a un caso real (de su propia elección) más allá del ejemplo de libro.
4. **Comparar** el patrón con alternativas (otros patrones, soluciones sin patrón) y argumentar cuándo conviene cada una.
5. **Comunicar** un concepto técnico complejo en 15 minutos a una audiencia técnica.

---

2. Modalidad

- Grupos de **máximo 5 alumnos**.
- Cada grupo presenta **1 o 2 patrones GoF**, de modo que entre todos cubran **los 23 patrones**.
- Distribución sugerida: 11 grupos × 2 patrones + 1 grupo × 1 patrón. La cátedra coordina o los grupos se autoorganizan (ver tabla de sugerencia en sección 6).
- **Caso conductor libre por grupo**: cada grupo elige un dominio propio (no es obligatorio ChangoMás — pueden ir a banca, salud, gaming, e-commerce, IoT, lo que prefieran).
- Tiempo por presentación: **15 minutos** + 5 minutos de Q&A del resto del curso y del docente.

> *El objetivo es que **entre todos los grupos** se cubra el catálogo completo de los 23 patrones GoF, generando un repositorio compartido del curso.*

---

3. Estructura obligatoria de cada presentación (15 min)

Cada presentación debe respetar la siguiente secuencia. Esto **es parte de la evaluación**:

Bloque	Min	Contenido obligatorio
1 · Intención	0-2	Definición del patrón en una frase + cita textual del libro GoF (1994). Familia a la que pertenece.
2 · Problema	2-6	Escenario sin el patrón aplicado: código / diagrama del problema, qué sale mal, por qué se vuelve insostenible al crecer.
3 · Solución	6-10	Diagrama de clases canónico del patrón + código resumido. Quién conoce a quién, cuál es la regla de dependencia.
4 · Caso propio	10-13	Aplicación al dominio elegido por el grupo. Mostrar código (puede ser pseudocódigo) y diagrama adaptado al caso.
5 · Cuándo NO usarlo	13-15	Trampas, anti-patterns relacionados, patrones que compiten, cuándo es sobreingeniería.

> ***No respetar la estructura penaliza la nota*** *porque parte del objetivo es que la audiencia pueda comparar patrones de forma uniforme.*

---

4. Entregables

Cada grupo entrega:

1. **Presentación** (PPTX o PDF) — la que va a usar en la exposición.
2. **1-pager por patrón** (1 carilla A4 por patrón presentado, en PDF) con: - Nombre + familia. - Intención (1-2 líneas). - Diagrama de clases canónico. - 1 ejemplo en código (15-25 líneas). - Cuándo SÍ usarlo / Cuándo NO usarlo. - 3 referencias bibliográficas (libros, papers, sitios estables) para profundizar.
3. **Repositorio del curso**: los 1-pagers de los 12 grupos se consolidan en un **PDF común** que queda como material de consulta para todos.

> *El 1-pager es **el aporte permanente** del grupo al resto de la cátedra.*

Reglas de entrega · OBLIGATORIAS

Estas reglas se aplican **sin excepciones**:

- **Entrega exclusivamente en aula virtual**, en la actividad correspondiente, **antes del inicio de la clase de presentación**. Una vez iniciada la clase se cierra la entrega.
- **Solo se puede presentar con documentos subidos** a la actividad del aula virtual. Si no está subido, no se presenta.
- **No se permite usar links a repositorios externos** (GitHub, GitLab, Bitbucket, Google Drive, Notion, etc.) durante la exposición. La presentación debe ser **autocontenida** en los archivos entregados.
- **Solo documentación de tipo invariante**: los archivos entregados (PDF, PPTX) deben ser **estáticos y autocontenidos**. No se aceptan:
- Links a Google Slides, Canva, Figma, Miro u otras herramientas online.
- Links a documentos vivos (Google Docs, Notion, HackMD).
- Embeds que requieran conexión a internet para visualizarse.
- Repositorios de código vivos (linkear código como referencia bibliográfica está OK, pero no como contenido de la presentación).
- Las **referencias bibliográficas** sí pueden citar URLs (Refactoring.Guru, libros, papers), porque son material de consulta posterior — no contenido necesario para la presentación.

Penalizaciones:

Situación	Consecuencia
Entrega después del inicio de clase	**No se presenta · 0 pts**
Presentación con link a Google Slides en vez de PDF/PPTX subido	**No se presenta · 0 pts**
Recurso en vivo (Codepen, Figma, etc.) que falla	Pierde el bloque correspondiente
Falta el 1-pager en PDF	-20 pts (pierde la categoría “Materiales” completa)

> ***Por qué tan estricto**: queremos que el material **sobreviva al curso**. En 2 años, cualquier alumno o docente debe poder abrir el PDF del repositorio del curso y entender el patrón sin depender de servicios externos que pueden caer, cerrarse o cambiar.*

---

5. Rúbrica de evaluación (100 pts)

Criterio	Pts	Qué se evalúa
Comprensión del patrón	25	¿Captaron la intención real? ¿Distinguen el patrón de otros similares (Strategy vs State, Adapter vs Facade)?
Caso propio aplicado	20	¿El ejemplo es **real y verosímil**? ¿No es un copy del libro? ¿El caso justifica el patrón?
Estructura de la presentación	15	¿Respetaron los 5 bloques obligatorios + el tiempo?
Materiales	15	Calidad de slides + diagramas + código + 1-pager. Legibilidad, claridad.
Cuándo NO usarlo	10	¿Identificaron las trampas reales? ¿Conocen los anti-patterns asociados?
Manejo de Q&A	10	¿Respondieron con criterio? ¿Reconocieron lo que no sabían?
Participación de TODO el grupo	5	Idealmente los 5 integrantes hablan (no solo uno expone).

Nota mínima de aprobación: 60/100. **Reentrega permitida**: con penalización del 20% si la nota es < 60.

---

6. Distribución sugerida de patrones

> *Los grupos pueden negociar la distribución entre ellos. Si no, esta es la asignación que la cátedra propone como **default balanceado** (mezcla intencionada de familias para forzar comparaciones).*

Grupo	Patrones asignados (sugerencia)	Combo educativo
G1	Singleton + Multiton (variante)	Creacional simple + variante
G2	Factory Method + Abstract Factory	Creacional escalonado
G3	Builder + Prototype	Creacional avanzado
G4	Adapter + Bridge	Confusión típica (los dos hablan de “compatibilidad”)
G5	Composite + Decorator	Los dos usan recursión / wrapping
G6	Facade + Proxy	Los dos “envuelven” pero distinto propósito
G7	Flyweight + Iterator	Optimizaciones que se confunden
G8	Strategy + Template Method	Los dos cambian comportamiento — sutil diferencia
G9	Observer + Mediator	Los dos coordinan, distinto enfoque
G10	Command + Memento	Los dos “guardan” cosas, distinto fin
G11	Chain of Responsibility + State	Los dos delegan, distinta lógica
G12	Visitor + Interpreter	Los dos avanzados, los más difíciles

> *La consigna: cuando son **dos** patrones, el grupo debe presentar los dos y además **mostrar la diferencia** entre ellos al final (1 minuto extra). Eso justifica el combo elegido.*

---

7. Caso conductor propio · qué se espera

Cada grupo elige su propio dominio. Ejemplos válidos (no obligatorios):

- **App de delivery** (Rappi, PedidosYa, Uber Eats)
- **Streaming de video** (Netflix, Twitch)
- **Plataforma de gaming** (matchmaking, microtransacciones, inventario)
- **App bancaria** (transferencias, plazos fijos, débito automático)
- **Sistema de salud** (turnos, historia clínica, recetas digitales)
- **IoT industrial** (sensores, alertas, predictivo)
- **Plataforma educativa** (LMS, cursos, evaluaciones)
- **E-commerce** (recomendaciones, carrito, checkout)
- **Red social** (feed, notificaciones, mensajería)
- **Sistema de transporte** (uber, scooter sharing, micros)

Lo que se espera del caso:

- Debe ser un dominio que **el grupo entienda** o pueda investigar.
- El **problema** que motiva el patrón debe ser **realista** dentro de ese dominio.
- No vale un caso forzado del estilo “imaginemos que tenemos que…” si no se conecta con un problema concreto.

---

8. Bibliografía y recursos online

Libros (los canónicos)

- **Gamma, Helm, Johnson, Vlissides** — *Design Patterns: Elements of Reusable Object-Oriented Software* (1994). El libro original. Inglés. Existe edición en español, busquen “Patrones de Diseño” de los mismos autores.
- **Freeman, E. & Robson, E.** — *Head First Design Patterns* (2da ed., 2020). Tratamiento didáctico, muy visual. Recomendado si nunca vieron patrones antes.
- **Larman, C.** — *UML y Patrones* (2da ed., 2004). Capítulos 23-26. Conecta los patrones GoF con los GRASP que ya vimos en D02.
- **Martin, R.** — *Agile Software Development: Principles, Patterns, and Practices*. Patrones con sabor SOLID.

Sitios web

Sitio	URL	Útil para
Refactoring.Guru	refactoring.guru/design-patterns	El mejor recurso visual gratuito. Diagramas + ejemplos en muchos lenguajes
SourceMaking	sourcemaking.com/design_patterns	Explicaciones largas, código C++/Java/PHP
The Catalog of Design Patterns	java-design-patterns.com	Implementaciones en Java de los 23 patrones
Microsoft Cloud Design Patterns	learn.microsoft.com/azure/architecture/patterns	Patrones más modernos (de nube), complementan los GoF
Wikipedia · Design pattern (object-oriented)	en.wikipedia.org/wiki/Software_design_pattern	Para definiciones rápidas

Repositorios GitHub recomendados

- `iluwatar/java-design-patterns` — implementaciones en Java de TODOS los patrones GoF + extras (15k+ stars).
- `kamranahmedse/design-patterns-for-humans` — guía concisa en JS / PHP estilo “para humanos”.
- `tcastroe/python-patterns` — patrones en Python.
- `oxnz/design-patterns` — implementaciones múltiples lenguajes.

Videos (cuando uno está atascado entendiendo un patrón)

- **Christopher Okhravi** (YouTube, canal en inglés): tiene una serie con 1 video por patrón. Claro y profundo.
- **Derek Banas** (YouTube): tutoriales rápidos en Java.
- **Mosh Hamedani**: cursos pagos pero muy buenos.

Para el caso propio (research del dominio)

- **martinfowler.com** — artículos sobre patrones aplicados a problemas reales de la industria.
- **HighScalability.com** — arquitecturas reales de empresas (Netflix, Spotify, Uber, etc.).
- **InfoQ.com** — papers y charlas sobre arquitectura y patrones.

9. Consejos prácticos

- **Empezar por la intención**, no por el código. Si no podés explicar en una frase qué problema resuelve el patrón, no lo entendiste.
- **Buscar ejemplos malos primero**. El patrón se aprecia cuando vemos el código sin él, no al revés.
- **Hacer el diagrama a mano antes que en herramienta**. Si entendiste el patrón, lo dibujás en 30 segundos en un papel.
- **Practicar la presentación cronometrada**. 15 minutos es **muy poco**. Vas a tener que cortar.
- **Preparar al menos 3 preguntas que esperan que les hagan** y ensayar respuestas.
- **Investigar la historia del patrón**. Casi todos tienen una historia interesante (ej.: Observer viene del Modelo MVC original de Smalltalk).
- **Comparar con otros patrones**. La parte de "Cuándo NO usarlo" es donde se separa el grupo que entendió del que solo leyó.

---

10. FAQ

¿Podemos usar IA (ChatGPT, Claude) para preparar? Sí, pero solo como herramienta de investigación / brainstorming. **Tienen que poder defender cualquier cosa que digan**. Si en las preguntas no se sabe defender, de nada sirve el uso de IA.

¿Podemos modificar el orden de la estructura obligatoria? No. Es parte de la evaluación que el curso pueda comparar patrones de forma uniforme. Si lo cambian, pierden los 15 pts de "estructura".

¿Y si entre los integrantes del grupo nos peleamos por quién expone qué? Idealmente los 5 integrantes hablan (esto da 5 pts en la rúbrica). Si solo expone 1, es nota baja.

¿Podemos cambiar el caso conductor a último momento? Sí, pero deben notificar al docente **antes de empezar la actividad**, no al momento de la presentación.

¿Y si me olvido de subir el PDF antes de la clase? No se presenta. La entrega cierra cuando arranca la clase. Sin excepciones (incluye problemas de internet, computadora rota, etc. — el archivo se sube con anticipación, no contra reloj).

¿Puedo mostrar código en vivo desde mi VSCode / GitHub durante la presentación? No. Todo lo que se muestra debe estar embebido en el PDF/PPTX subido al aula virtual. Capturas de código sí, código en vivo no.

¿Puedo linkear a Refactoring.Guru en la bibliografía del 1-pager? Sí — eso es **referencia bibliográfica**, no contenido necesario para la presentación. Es como citar un libro: el lector va si quiere.

¿Y si mi presentación incluye un video de YouTube? Mejor un screenshot + cita del video en la bibliografía. Reproducir video en vivo asume que hay internet y que el video sigue disponible — falla la regla de "invariante".

¿Hay extra para el primer y mejor grupo? Sí: el mejor 1-pager según el resto del curso (votación al final) **abre como referencia oficial del repositorio del curso**.

Material complementario de la clase D03 · Patrones GoF. Cualquier duda sobre la consigna, llevar a clase o consultar por aula virtual.

— Ing. Fernando Bono · MyD 2026 · UNC-ESCMB-AUSI